Reducing Computational Redundancy in the Hardware Architecture of a 2-D BCH Decoder over GF(16)

August 17, 2026

Abstract

The 2-D inverse discrete Fourier–Fermat transform (2-D IDFFFT) used to decode a 15×15 2-D BCH code over GF(16) requires, at every one of the 225 output points, a field multiplication between each spectral coefficient and a corresponding twiddle factor. Because the spectrum is organised into conjugate (cyclotomic) classes and GF(16) has only 16 elements, many of these 225 multiplications are, in fact, identical. This note formalises that redundancy, classifies every conjugate class by the number of *distinct* field multiplications it actually requires (its *computation class*), and uses this classification to propose a re-architected decoder that computes each distinct product once and routes it to all points that need it via an XOR fan-out network, in place of the current design's exhaustive, per-point multiply-and-XOR structure. Realised across the full 59-class, 225-point spectrum, the baseline design's 239 AND/239 XOR gates per output point total 53,775 AND and 53,775 XOR gates, against 3071 AND and 2460 XOR gates for the proposed architecture – a reduction of roughly 17.5× in AND gates and 21.9× in XOR gates.

1 Introduction

A binary $n \times n$ 2-D BCH code is constructed in the frequency domain over an extension field GF(2^m), where m is chosen so that $2^m = n + 1$. A $2t \times 2t$ block of the spectrum, together with all its conjugate positions, is forced to zero to guarantee a quasi-cyclic $t \times t$ burst error-correcting capability; the remaining spectral positions carry the message. The codeword is obtained from the encoded spectrum through a 2-D inverse discrete Fourier–Fermat transform (2-D IDFFFT). For the $n = 15$, GF(16) case with $t = 2$, this construction produces the 59-class conjugate structure summarised in Section 2.

2 Background: Spectral Structure of the 15×15 2-D BCH Code

2.1 Conjugate (Cyclotomic) Classes

Each cell of the 15×15 spectrum is indexed by an exponent pair (j, j'), $j, j' \in \{0, 1, \ldots, 14\}$, representing the root $\alpha^j \alpha'^{j'}$ of the code's generator, where α, α' are primitive 15th roots of unity in GF(16)*. Since the field has characteristic 2, the Frobenius map $x \mapsto x^2$ acts on an exponent pair by simultaneous doubling modulo 15:

$$(j, j') \longmapsto (2j \bmod 15,\ 2j' \bmod 15). \tag{1}$$

Repeated application of (1) to a pair (j, j') generates its *conjugate class* (orbit). Because $2^4 = 16 \equiv 1 \pmod{15}$, the Frobenius automorphism of GF(16) has order 4, so every orbit closes after at most four steps, giving class sizes of 4, 2, or 1 depending on $\gcd(j, 15)$ and $\gcd(j', 15)$.

Applying (1) to all 225 pairs $(j, j') \in \{0, \ldots, 14\}^2$ partitions the spectrum into exactly 59 classes, as summarised in Table 1.

Table 1: Conjugate class sizes for the 15×15 spectrum over GF(16).

Size	Count of classes	Points covered	Classes
1	1	1	C-1
2	4	8	C-2, C-3, C-4, C-5
4	54	216	C-a … C-k (11), C-6 … C-48 (43)
Total	59	225	

For the specific 15×15, $t = 2$ code, the 44 spectral positions forced to zero (the "spectral nulls") fall into 11 conjugate classes, labelled C-a through C-k. The remaining 181 positions, which carry the message, are distributed over the other 48 classes: 1 class of size 1 (C-1), 4 classes of size 2 (C-2 – C-5), and 43 classes of size 4 (C-6 – C-48).

3 Baseline (Current) Decoder Architecture

The current decoder evaluates the 2-D IDFFFT directly, point by point:

1. Divide the incoming spectral values according to their conjugate classes.
2. For every one of the $n \times n = 225$ points, use the conjugate class value to calculate the corresponding insertion (twiddle-multiplied) term at that point.
3. XOR the contributing terms independently at each point to form the output.

As currently implemented, this direct evaluation costs **239 AND gates and 239 XOR gates per output point**: each of the 225 points independently re-derives its contribution from every conjugate class via a fresh GF(16) multiplication (an AND/XOR field-multiplier instance), even though, as shown next, very few *distinct* products are actually involved. Since this per-point cost is paid independently at all $n \times n = 225$ output points, the full decoder's baseline gate count is

$$239 \times 225 = 53{,}775 \text{ AND gates and } 53{,}775 \text{ XOR gates.} \tag{2}$$

4 Motivation: Redundant Field Multiplications

4.1 The Core Observation

Writing the 2-D IDFFFT as

$$c_{i,i'} = \sum_{j=0}^{n-1} \sum_{j'=0}^{n-1} \alpha^{-(ij+i'j')}\, C_{j,j'}, \tag{3}$$

each output point (i, i') requires multiplying every spectral coefficient $C_{j,j'}$ by a twiddle factor $\widehat{C}_{j,j'} = \alpha^{-(ij+i'j')}$. Collecting the Hadamard product $\overline{C}_{j,j'} = C_{j,j'} \cdot \widehat{C}_{j,j'}$ lets (3) be rewritten as a plain sum,

$$c_{i,i'} = \sum_{j=0}^{n-1} \sum_{j'=0}^{n-1} \overline{C}_{j,j'}. \tag{4}$$

Since a fixed insertion-point value is repeated (via the same Frobenius conjugacy) across every one of the 225 output points, and since GF(16) has only 16 possible values, the multiplier in (4) is being re-evaluated on the same handful of field elements over and over. The number of *distinct* field elements a given conjugate class ever needs to multiply by, across the full 225-point sweep, is generally far smaller than 225 – as small as 1.

5 Computation Classes: Classifying the Redundancy

5.1 Definition

For a conjugate class with insertion point (J, J'), define its **computation class** as the number of *distinct, nonzero-count* powers of α that occur among the 225 values produced when the class is swept across every output point (i, i'), $i, i' \in \{0, \ldots, 14\}$. This count is always a divisor of 225 and, empirically, takes only the values 15, 5, 3, or 1 – i.e. the multiplication that step 2 of the current decoder (Section 3) instantiates independently at 225 points in fact only ever needs 15, 5, 3, or 1 distinct GF(16) multiplier evaluations. The four cases below are representative examples, one per computation class size, each verified over the full 15×15 sweep.

5.2 Case Study: 15-Computation Class (C-18)

Insertion point $(J, J') = (3, 5)$, class size 4.

Table 2: Point values (powers of α) generated by class C-18 across all 225 output points.

$i \backslash i'$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0
1	7	2	12	7	2	12	7	2	12	7	2	12	7	2	12
2	4	14	9	4	14	9	4	14	9	4	14	9	4	14	9
3	1	11	6	1	11	6	1	11	6	1	11	6	1	11	6
4	13	8	3	13	8	3	13	8	3	13	8	3	13	8	3
5	10	5	0	10	5	0	10	5	0	10	5	0	10	5	0
6	7	2	12	7	2	12	7	2	12	7	2	12	7	2	12
7	4	14	9	4	14	9	4	14	9	4	14	9	4	14	9
8	1	11	6	1	11	6	1	11	6	1	11	6	1	11	6
9	13	8	3	13	8	3	13	8	3	13	8	3	13	8	3
10	10	5	0	10	5	0	10	5	0	10	5	0	10	5	0
11	7	2	12	7	2	12	7	2	12	7	2	12	7	2	12
12	4	14	9	4	14	9	4	14	9	4	14	9	4	14	9
13	1	11	6	1	11	6	1	11	6	1	11	6	1	11	6
14	13	8	3	13	8	3	13	8	3	13	8	3	13	8	3
0	10	5	0	10	5	0	10	5	0	10	5	0	10	5	0

Table 3: Occurrence count of each power of α for C-18: all 15 nonzero powers appear, each exactly 15 times.

Power of α	Number of occurrences
0–14 (all 15 powers)	15 each

Here all 15 nonzero elements of GF(16) appear, each exactly 15 times ($15 \times 15 = 225$). The current architecture would instantiate 225 multiplier evaluations for this class; only 15 distinct ones are actually required.

5.3 Case Study: 5-Computation Class (C-7)

Insertion point $(J, J') = (0, 3)$, class size 4.

Table 4: Point values for class C-7 – every row is identical.

$i \backslash i'$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0
1,	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
2	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
3	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
4	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
5	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
6	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
7	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
8	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
9	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
10	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
11	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
12	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
13	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
14	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
0	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0

Only the powers $\{0, 3, 6, 9, 12\}$ ever appear, each 45 times ($5 \times 45 = 225$): five distinct multiplier evaluations replace 225.

Table 5: Occurrence count of each power of α for C-7.

Power	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Count	45	0	0	45	0	0	45	0	0	45	0	0	45	0	0

5.4 Case Study: 3-Computation Class (C-3)

Insertion point $(J, J') = (5, 0)$, class size 2.

Table 6: Point values for class C-3 – every value in a row is constant, and rows cycle with period 3.

i	Value (constant across all i')
1	10
2	5
3	0
4	10
5	5
6	0
7	10
8	5
9	0
10	10
11	5
12	0
13	10
14	5
0	0

Table 7: Occurrence count of each power of α for C-3.

Power	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Count	75	0	0	0	0	75	0	0	0	0	75	0	0	0	0

Only the powers $\{0, 5, 10\}$ ever appear, each 75 times ($3 \times 75 = 225$): three distinct multiplier evaluations replace 225.

5.5 Case Study: 1-Computation Class (C-1)

Insertion point $(J, J') = (0, 0)$, class size 1.

C-1 always contributes 0: it needs no multiplier at all, only a trivial route/short-circuit.

Table 8: Point values for class C-1: identically zero at every point.

Power of α	Number of occurrences
0 (i.e. the value 0)	225
all other powers	0

5.6 Discussion

Across these four examples the pattern is exact: the number of distinct GF(16) elements a conjugate class' contribution ever takes, over the full 225-point sweep, is 15, 5, 3, or 1 – always a divisor of $225 = 15^2$, and the occurrence count of each distinct value is always 225 divided by the computation-class size ($15, 45, 75, 225$ respectively). Every one of the 59 conjugate classes of the 15×15 spectrum falls into one of these four computation classes, so the true multiplicative work of the transform is bounded by 15 evaluations per class rather than 225 – a reduction of up to $15\times$ per class before any further sharing between classes is even considered.

6 Proposed (Optimized) Decoder Architecture

6.1 Algorithm

The redundancy identified above motivates a decoder that computes each distinct product once per conjugate class and reuses it, rather than recomputing it at every point:

1. Divide the incoming messages according to conjugate classes, as before.
2. For each conjugate class, compute its product with only the (at most 15, and often just 5, 3, or 1) distinct elements of GF(16) dictated by its computation class, rather than with all 225 points individually.
3. XOR these precomputed products according to the routing each output point requires – i.e., the product of a conjugate class' insertion-point value with a given GF(16) element is fanned out over an XOR network to every point (i, i') where that particular product is needed.

6.2 Explanation of the Architecture

In the baseline design (Section 3), the multiply stage and the XOR-combine stage are both instantiated 225 times per conjugate class, regardless of how many of those 225 products are actually distinct. The proposed design factors this into two stages that match the true computational content shown in Section 5:

6.3 Architecture Diagram

- The **multiplier bank** is sized to each class's computation class rather than to 225: a class such as C-1 needs no multiplier (constant 0), C-3-type classes need 3 multiplier evaluations, C-7-type classes need 5, and classes such as C-18 need the full 15.
- The **routing/XOR network** takes over the job of reproducing the 225-point pattern: since Theorem 1 (Section 4) guarantees each product recurs at a predictable, fixed set of points, that set can be hard-wired as a fan-out rather than re-derived by a fresh multiplication.

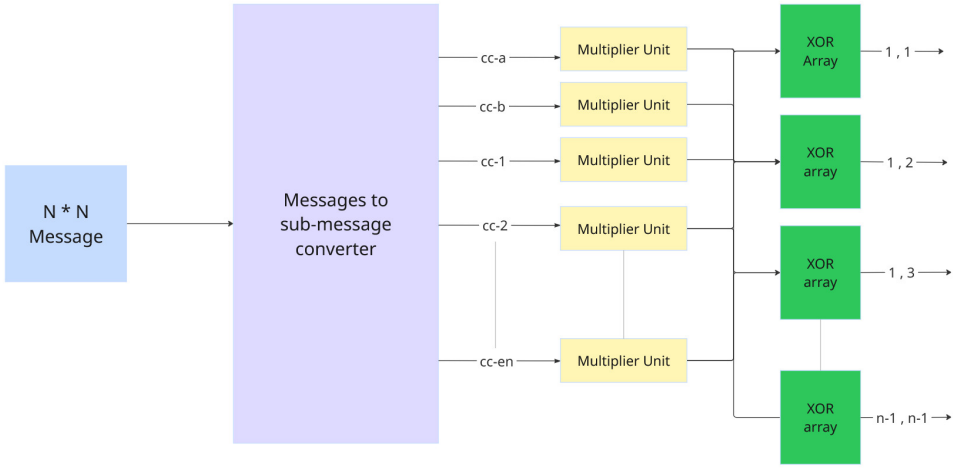


Figure 1: Proposed Architecture Diagram

6.4 Complexity Reduction for Size-4 and Size-2 Classes

For message encoding, the 48 non-null conjugate classes consist of 1 class of size 1, 4 of size 2, and 43 of size 4. For a size-4 class, if $P_1(\alpha) = a_3\alpha^3 + a_2\alpha^2 + a_1\alpha + a_0$ and $P_2(\alpha) = b_3\alpha^3 + b_2\alpha^2 + b_1\alpha + b_0$ are the polynomial representations of the class value and the corresponding twiddle factor, the MSB of the sum of the polynomial product $P_1(\alpha)P_2(\alpha)$ over the class' four points simplifies to

$$\text{out}^{\text{MSB}}_{\text{size 4}} = c_0 \oplus (c_0 \oplus c_2) \oplus (c_0 \oplus c_1 \oplus c_2 \oplus c_3) \oplus (c_0 \oplus c_1) = c_3 = a_0b_3 \oplus a_1b_2 \oplus a_2b_1 \oplus a_3b_0 \oplus a_3b_3. \quad (5)$$

Similarly, for a size-2 class the MSB of the summed product reduces to

$$\text{out}^{\text{MSB}}_{\text{size 2}} = c_0 \oplus (c_0 \oplus c_2) = c_2 = a_0b_2 \oplus a_1b_1 \oplus a_2b_0 \oplus a_2b_3 \oplus a_3b_2 \oplus a_3b_3. \quad (6)$$

Equations (5)–(6) show that the output bit that would otherwise require summing four (or two) independently multiplied points collapses to a fixed XOR expression in the bits of the two operands. This is exactly the kind of structural shortcut the proposed architecture is built to exploit: because the class structure is preserved under the twiddle multiplication, this closed-form bit expression can be implemented once per class and reused, rather than being re-derived per-point.

7 Expected Gains

Section 5 showed that every one of the 59 conjugate classes falls into exactly one of four computation-class types, requiring only 15, 5, 3, or 1 distinct GF(16) multiplications instead of 225. Realising the proposed architecture (Section 6) means building one dedicated reduced-multiplier unit, sized to its computation-class type, for every *message-carrying* conjugate class – the 11 spectral-null classes need no hardware at all, since their value is identically zero at every point. Table 9 gives the resulting gate count, broken down by computation-class type.

The *Classes* column reproduces the 59-class spectral partition of Table 1; the *Spectral nulls* column is the number of those classes, within each computation-class type, that belong to the 11 null classes C-a – C-k; and *Message-carrying classes* is the remainder, which sums to $38 + 5 + 4 + 1 = 48$, matching the 48 classes that actually carry the encoded message (Section 2). The *AND/class* and *XOR/class* columns are the gate cost of the single reduced-multiplier unit built for one class of that

Table 9: Realised AND/XOR gate cost of the proposed architecture, by computation-class type.

Computation class	Classes (total)	Spectral nulls	Message-carrying classes	AND /class	XOR /class	Total AND gates	Total XOR gates
15 Computations	48	10	38	75	60	2850	2280
5 Computations	6	1	5	25	20	125	100
3 Computations	4	0	4	24	20	96	80
1 Computation	1	0	1	0	0	0	0
Total	**59**	**11**	**48**	–	–	**3071**	**2460**

type – e.g. the 15-computation unit realises all 15 distinct constant multiplications at 75 AND and 60 XOR gates (5 AND and 4 XOR gates per distinct multiplication), while the 1-computation type needs no multiplier at all. Since only message-carrying classes are instantiated in hardware, the overall total is

$$\text{Total gates} \;=\; \sum_{\text{type}} \big(\text{classes} - \text{spectral nulls}\big) \times \big(\text{gates/class}\big), \tag{7}$$

which gives $38{\times}75 + 5{\times}25 + 4{\times}24 + 1{\times}0 = 3071$ AND gates and $38{\times}60 + 5{\times}20 + 4{\times}20 + 1{\times}0 = 2460$ XOR gates for the complete decoder – exactly the totals in Table 9.

Two observations follow directly from this breakdown. First, the 11 spectral-null classes contribute nothing to the gate count: they are filtered out for free by the classification itself, rather than being multiplied out and then discarded as in the baseline design. Second, the 38 message-carrying classes of the 15-computation type dominate the total, accounting for 2850 of 3071 AND gates and 2280 of 2460 XOR gates (roughly 93% of each) – so this type is where any further optimization effort would have the largest payoff. It is also worth noting that the per-multiplication cost is fairly uniform for the two largest categories ($75/15 = 5$ AND gates and $60/15 = 4$ XOR gates per multiplication for the 15-computation type; $25/5 = 5$ AND and $20/5 = 4$ XOR per multiplication for the 5-computation type), while the 3-computation type is comparatively less efficient per multiplication ($24/3 = 8$ AND gates), consistent with there being less structure to share when only a few distinct products are involved.

These totals represent the fully realised, class-by-class hardware cost of the proposed architecture across the entire 15×15 spectrum – i.e. all 59 classes routed to all 225 output points, since each class' product is now computed once and then wired to every point that needs it. This is directly comparable to the baseline total derived in Section 3: the current decoder pays its 239 AND and 239 XOR gates independently at *each* of the 225 points, for a full-decoder cost of $239 \times 225 = 53{,}775$ AND gates and 53,775 XOR gates (Eq. (2)). Against this baseline, the proposed architecture's 3071 AND and 2460 XOR gates represent a reduction of

$$\frac{53{,}775}{3071} \approx 17.5\times \text{ fewer AND gates}, \qquad \frac{53{,}775}{2460} \approx 21.9\times \text{ fewer XOR gates}. \tag{8}$$

This large gain is exactly what the compute-once, route-many restructuring was designed to capture: the baseline pays the full 225-point multiplier cost independently at every point, while the proposed design pays for each distinct product only once per class and reuses it via the XOR routing network – so the saving scales with how much repetition existed in the first place, which Section 5 showed to be substantial for every computation-class type.

8 Conclusion

By recognising that the 15×15, GF(16) spectrum's 59 conjugate classes each only ever multiply against a small, fixed set of distinct field elements – 1, 3, 5, or 15 of them, never more – the 2-D BCH decoder's hardware can be restructured from an exhaustive per-point multiply-and-XOR scheme into a compute-once, route-many scheme. The four computation-class case studies (C-1, C-3, C-7, C-18) span the full range of redundancy observed across all 225 spectral positions, and the size-4/size-2 closed-form reductions of Section 6 give a concrete, bit-level route to realising this saving in hardware. Measured over the complete decoder – all 59 classes routed to all 225 output points – this restructuring takes the baseline's 53,775 AND and 53,775 XOR gates down to 3071 AND and 2460 XOR gates: roughly a 17.5× reduction in AND gates and a 21.9× reduction in XOR gates.